

Injecting Externally Encoded Media into WebRTC Peer Connections

Design and Implementation of the Encoded Source API in Chromium
and WebRTC

Leonardo Evi

Internship report, Master's degree, Sorbonne University, Paris

Codebases:

- WebRTC (webrtc.googlesource.com/src)
- Chromium (chromium.googlesource.com/chromium/src)

Contents

1	Introduction	2
2	The Web Platform	2
3	Chromium	3
4	WebRTC	6
5	Digital Audio and Video Encoding	9
6	Limitations of the Current Web Platform	11
7	Use Cases	12
8	Proposed Solution	13
9	Demonstration	16
10	Conclusion and Future Work	17
A	Glossary	18
B	List of Submitted Code Changes	19
C	References	20

1 Introduction

Video calls conducted entirely inside a browser tab are one of the more visible achievements of the modern Web Platform: no plugin to install, no separate application to open, just a page that a camera and a microphone can talk through to someone else, anywhere. This is made possible by WebRTC, a set of protocols and browser programming interfaces, built into every major browser, that lets two machines establish a connection and exchange audio and video directly between them, in real time.

Compressing that audio and video, so that it fits over an ordinary internet connection, is something the browser has always done for the application, using its own built-in encoder, and it does this well. What a web application cannot do today is compress that media itself and hand the result to the browser instead, through a standardized interface: an `RTCPeerConnection`, the object at the center of a WebRTC call, only ever accepts a raw stream captured from a camera or microphone, and only the browser's internal encoder is allowed to feed it.

This internship implemented that missing capability. Working across both open-source repositories that make up a browser's real-time stack, WebRTC and Chromium, it added a way for a web application to construct a compressed audio or video frame from scratch and inject it into a live peer connection, while still receiving the same feedback signals, key-frame requests and bandwidth updates, that a real encoder depends on to keep a stream usable. The result was validated with a working demonstration application, and the design is documented in a public explainer now moving through standardization at the W3C.

2 The Web Platform

A web page is fetched from a server as a document, but since the mid-1990s that document may also contain a program, written in JavaScript, which the browser executes on the user's machine. Over three decades this has grown into something much larger: the browser exposes to that program a very extensive library of capabilities: drawing, audio, storage, cryptography, camera and microphone access, GPU access, and several kinds of networking. Compiled languages such as C++ and Rust can also be targeted at the browser through **WebAssembly (WASM)**, a portable binary instruction format that runs at close to native speed inside the same sandbox. The union of these capabilities is what the industry calls the **Web Platform**. Video conferencing, collaborative editing, cloud gaming, remote desktop, and live broadcasting are all built on it: what the platform can and cannot do sets an upper bound on what can be built without asking users to install native software.

Two properties define this platform. First, the code running on it is untrusted: a user visits a URL and the browser immediately runs code written by a

stranger, so every capability has to be designed under the assumption that the application in front of it may be hostile. This assumption shapes a number of design decisions described later in this report that would otherwise look like unnecessary indirection. Second, the platform is not owned by any single company. It is specified by independent standards bodies and then implemented separately by several competing browser engines, which must agree closely enough that a page written once works everywhere. Two bodies matter here:

The **World Wide Web Consortium (W3C)** [1] specifies the programming interfaces that applications see: the names of the objects and methods a JavaScript program can call, written down in an interface language called **WebIDL**. The relevant group for this report is the WebRTC Working Group, which maintains interfaces such as `RTCPeerConnection`. At this level, the contribution described in this report is a single new method along these lines:

```
partial interface RTCRtpSender {  
  Promise<undefined> createEncodedSource(Worker worker);  
};
```

Section 8 explains what this method does and why; for now, it is enough to know that naming a method, its arguments and what it returns, like this, is exactly the kind of thing the W3C is responsible for.

The **Internet Engineering Task Force (IETF)** [2] specifies the wire protocols instead: the actual bytes that travel between two machines, independent of any particular browser. The protocols used by WebRTC, introduced in Section 4, are IETF standards. This report's contribution sits entirely on the W3C side: it adds a new interface for an application to use, without changing anything about what travels over the network.

A new browser capability follows a characteristic life cycle: an *explainer* document motivating the feature and sketching the interface; discussion in the relevant working group; an experimental implementation in at least one browser engine; refinement of the design in light of that implementation; a formal specification; and, eventually, implementation by the other engines. This internship's contribution sits in the middle of that cycle: an experimental implementation, and the design feedback that came from building it.

3 Chromium

Chromium is the open-source browser project that underlies several shipping browsers, including Google Chrome, Microsoft Edge, Opera, and Brave. It is developed in the open at `chromium.googlesource.com` under a permissive open-source license, and while most of its contributions come from engineers employed by Google, its review process does not distinguish between them and anyone else: every change is proposed as a patch and reviewed on `chromium-review.googlesource.com` by an owner of the affected area of code,

and it is merged only once that reviewer approves it. This is the process by which the Chromium-side changes listed in Appendix B were merged during this internship.

Chromium’s own architecture reflects the same untrusted-code assumption that shapes the rest of the Web Platform (Section 2), applied internally: the browser is split into several operating-system processes rather than run as one program, a design documented in Chromium’s own *Multi-process Architecture* reference [14]. A privileged **browser process** owns the parts of the system a website should never touch directly: the filesystem, raw network sockets, camera and microphone hardware, the window and its UI chrome. Each open site instead runs inside its own **renderer process**, which is confined by an operating-system sandbox (seccomp-bpf on Linux, Seatbelt on macOS, restricted job objects on Windows) that denies it direct access to the filesystem or the network. A separate **GPU process** mediates access to graphics and video hardware. Processes never share memory or call into each other directly; every cross-process interaction goes through a narrow, versioned interprocess-communication layer called **Mojo** [16], so that a renderer can only ask the browser process to do something on its behalf, never do it itself, as illustrated in Figure 1.

This separation is not incidental; it follows from a documented internal principle, described in Chromium’s *The Rule of 2* [15] and illustrated in Figure 2: code that (1) parses untrustworthy input, (2) is written in a memory-unsafe language such as C++, and (3) runs with high privilege, should satisfy at most two of the three, typically by moving the third, privilege, behind a sandbox boundary. **Blink** [17], where this internship’s contribution lives, sits on the wrong side of that rule for the first two: it runs whatever JavaScript an untrusted web page hands it, in C++. This is only acceptable because it runs inside the sandboxed renderer process rather than with the browser process’s privileges. Exposing a new capability at this boundary, as the Encoded Source API does, does not weaken that guarantee: the renderer never needs elevated privilege to use it, and the bytes it hands to WebRTC are treated by the native pipeline exactly as untrusted as any other input arriving from a script, which is why the feature could be implemented entirely inside the existing sandbox rather than by carving out a new exception to it.

Two parts of Chromium’s internals matter for this report. **Blink** is the layer that implements the Web Platform interfaces introduced in Section 2: the C++ code backing JavaScript objects such as `RTCPeerConnection` is Blink code, and it is the boundary this internship’s contribution had to cross to reach the browser’s internal media pipeline. **V8** is Chromium’s JavaScript engine [18], the C++ program that actually parses and runs the application’s script, independently of Blink. A WebIDL interface like the one in Section 2 is not, by itself, executable: Blink’s build pipeline compiles each such interface into generated C++ “binding” code [19] that registers the corresponding object and method with V8, so that a JavaScript call such as `sender.createEncodedSource()` is routed by V8 into

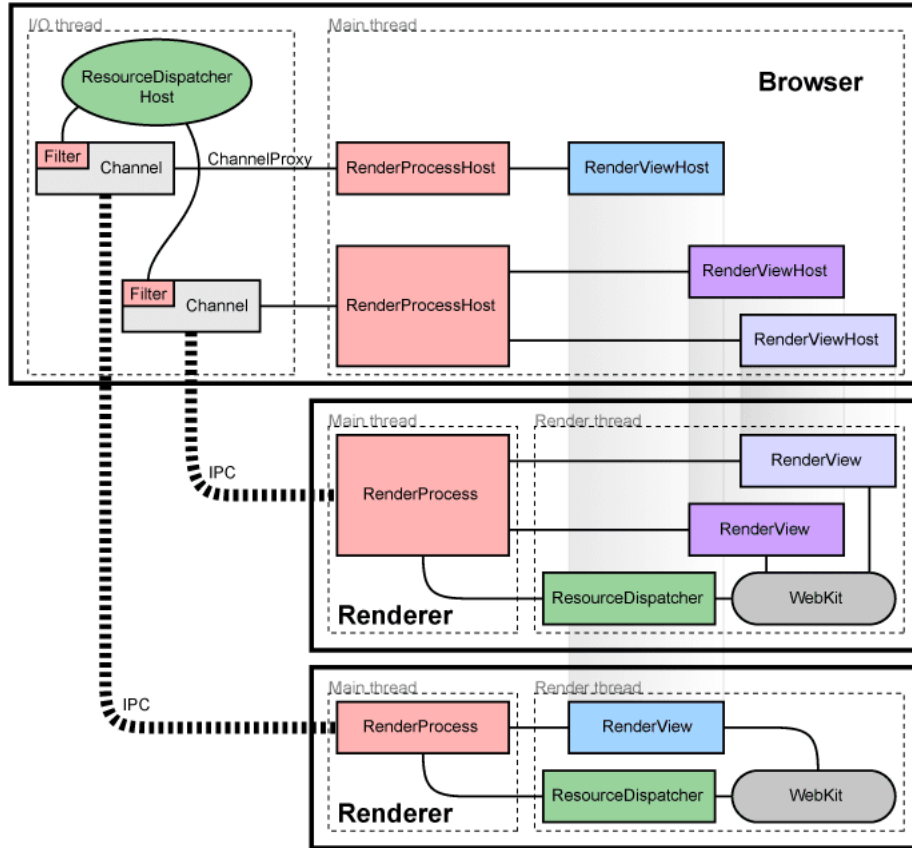


Figure 1: Chromium’s sandboxed process architecture. A privileged broker (the browser process) holds a policy engine and mediates every request coming from its sandboxed targets (renderer processes); today, that IPC channel is implemented by Mojo. The rendering engine running inside each of these targets was originally Apple’s WebKit, which Chromium later forked into its own engine, Blink. Source: [14].

the hand-written Blink implementation behind it. This project did not modify V8 itself, but adding a method to the WebIDL is what causes this generated glue code to exist in the first place.

Chromium also has its own process for shipping a new capability, separate from the W3C standardization life cycle described in Section 2 but coordinated with it: a feature is first proposed internally as an “Intent to Prototype”, built behind an experimental flag invisible to ordinary users, and only later, once its design has stabilized, proposed as an “Intent to Ship”: a public announcement inviting the other browser vendors to comment before the feature is turned on by default for everyone. The work described in this report currently sits at the prototype stage: a working, flag-gated implementation, developed in parallel with the explainer under discussion in the W3C WebRTC Working Group.

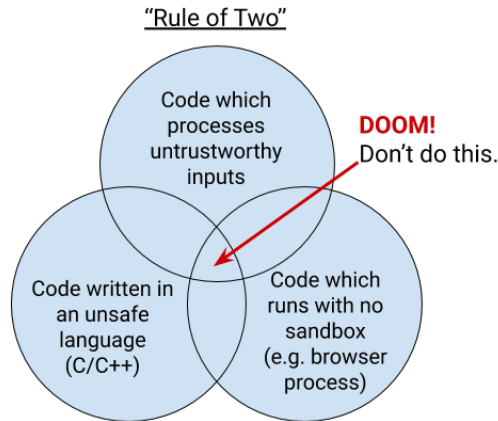


Figure 2: Chromium’s Rule of Two [15].

Real-time media communication itself, however, is not implemented inside Chromium directly: that responsibility is delegated to a standalone C++ library, simply named WebRTC, which is developed and reviewed independently of Chromium (at its own repository, webrtc.googlesource.com, under its own review process) and is the subject of the next section. Although the W3C and IETF standards described in Section 2 do not mandate any particular implementation, this same library is not unique to Chromium either: other browser engines, including Safari’s WebKit, bundle it too, which makes it a de facto shared foundation for real-time communication on the web rather than something each browser reimplements from scratch. Chromium keeps its own copy up to date through an automated process called a **roll**, which periodically pulls the latest WebRTC commit into its source tree; because this internship’s contribution touches both repositories, each of its Chromium-side changes had to wait for the corresponding WebRTC-side change to be merged and picked up by a roll before it could, in turn, be merged into Chromium.

4 WebRTC

WebRTC is a set of protocols, together with a core object called a *peer connection*, that let two ends exchange audio, video and data directly, in real time, with end-to-end encryption. A peer connection is not something unique to the browser: it is what the WebRTC library discussed in Section 3 exposes directly to native and server-side applications, and it is what the browser exposes to JavaScript, as the `RTCPeerConnection` object. *WebRTC for the Curious* [4], an open community reference on the protocol suite, is a useful companion to the following summary, that explains the main features that WebRTC implements. Figure 3 sketches the shape of this object before going into the detail of each feature.

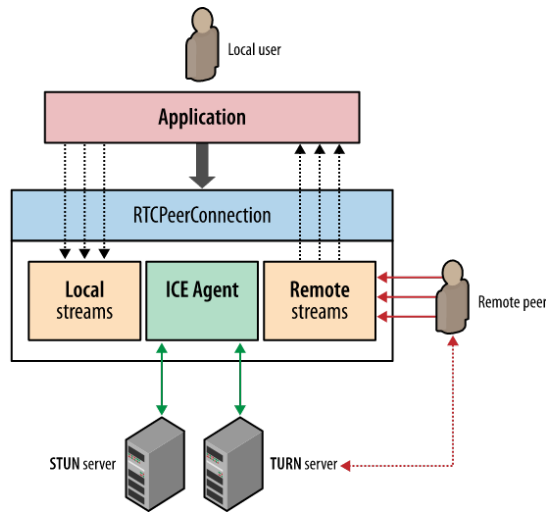


Figure 3: The `RTCPeerConnection` object, as the application sees it: it holds the local and remote streams exchanged with the peer, and an ICE agent that finds a usable network path with the help of STUN and TURN servers. Source: [5].

Signaling. Before any media flows, the two ends need to agree on what they are about to exchange: which media kinds, which codecs, in which order of preference, and which network addresses to try. This is done by exchanging a **Session Description Protocol (SDP)** document each way: one side sends an *offer*, the other replies with an *answer*. WebRTC deliberately does not standardize how this exchange itself happens; the application is free to carry these SDP documents over a WebSocket, an HTTP request, or any channel it already has, which is what lets WebRTC fit into any existing server infrastructure.

Connectivity: ICE. Most machines do not have a public, directly reachable network address: they sit behind a home router or a corporate firewall performing Network Address Translation (NAT). To find a path that actually works, each side gathers a list of **candidates** (address-and-port pairs it might be reachable on) of three kinds: its own local address (a *host* candidate), the address a STUN server observed it connecting from, which is often the router’s public address (a *server-reflexive* candidate), and, as a last resort, the address of a TURN server willing to relay traffic on its behalf (a *relayed* candidate). Both ends exchange their candidate lists and then try pairs of them until one is found that works end to end; this whole process is called **ICE**, and it is re-run automatically if the network changes, for example when a laptop moves from Wi-Fi to a mobile connection.

Encryption. Once a candidate pair is selected, the two ends encrypt the connection before anything else travels over it, using **DTLS** [10]: the same idea as the TLS that already secures ordinary web traffic over HTTPS, adapted

to run over UDP, since plain TLS assumes packets always arrive, in order, which UDP does not guarantee. Securing a connection this way normally also means checking who is on the other end, which HTTPS does by consulting a certificate authority, but WebRTC has no such authority to ask; instead, the signaling exchange described above already carried a fingerprint of each side’s certificate inside the SDP, and each side simply checks that the certificate presented during the DTLS handshake matches the fingerprint it was already given. The cryptographic keys this handshake produces are then used to key **SRTP** [11], which encrypts and authenticates every media packet that follows.

Packetization: RTP. Media itself travels as a sequence of **RTP** packets, defined by a deliberately minimal protocol: each packet carries a small header followed by an opaque, codec-specific payload that RTP itself does not interpret. The header records a *payload type* (identifying which codec the payload was encoded with), a monotonically increasing *sequence number* (so the receiver can detect loss and reordering), a *timestamp* (so it can play frames back at the right rate and keep audio and video in sync), and a *synchronization source (SSRC)*, a randomly chosen identifier that lets several independent streams (an audio track and a video track, say) share the same connection without their packets being confused for one another. RTP is deliberately unopinionated about a packet that never arrives: it does not retransmit anything itself, leaving loss to be concealed by the receiver or recovered through the RTCP feedback discussed next. A single compressed video or audio frame produced by an encoder is often too large for one packet and is split across several RTP packets sharing the same timestamp, with a *marker bit* in the header of the last one signaling that a frame boundary has been reached, as shown in Figure 4.

Feedback: RTCP. Running alongside RTP is a parallel control channel, **RTCP**, on which the receiving side continuously reports back to the sender. This feedback comes in two flavors that matter for this report. The first is *reactive*: if a receiver notices a small number of packets are missing, it can ask for them to be retransmitted (a **NACK**), but once enough loss accumulates that its decoder can no longer make sense of the stream, retransmission stops being useful, and it instead sends a **key-frame request** (called PLI or FIR in the protocol), asking the sender to start over with a frame that does not depend on anything already lost. The second is *proactive*: the two ends continuously exchange timing information that lets the sender estimate how much bandwidth the network path can currently sustain. An algorithm such as Google Congestion Control (GCC) does this by watching how the delay between consecutive packets grows or shrinks, and the sender turns that estimate into a bitrate budget it hands to its encoder. Together, a key-frame request and a bandwidth estimate are the feedback loop that turns a raw encoder into something usable for real-time media: without them, a video call cannot recover from packet loss or adapt to a changing network.

Today, this entire pipeline runs inside the browser: a browser-supplied encoder produces the outgoing frames, and that same internal code is what receives key-

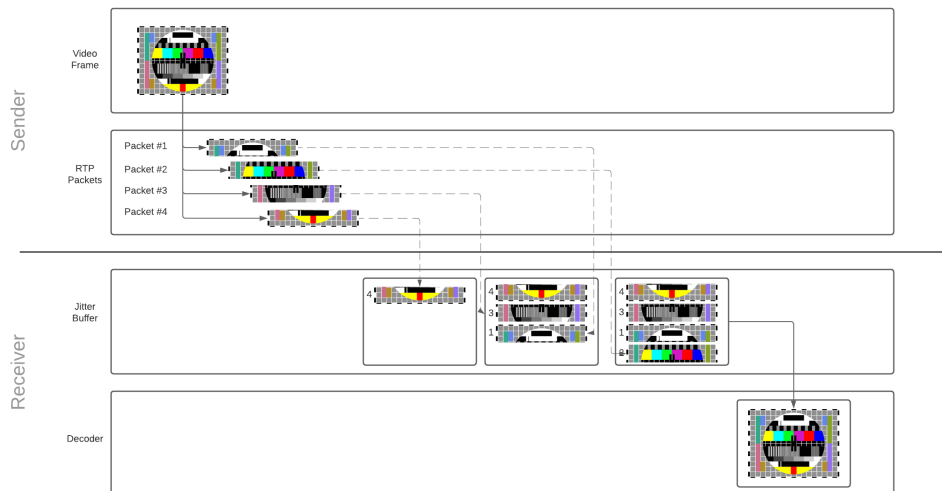


Figure 4: A single encoded video frame, packetized by the sender into several RTP packets and reassembled by the receiver. Packets can arrive out of order; as far as this report is concerned, it is enough to know that the receiver briefly holds incoming packets in a jitter buffer, which puts them back in the right order before the reassembled frame is handed to the decoder. Source: [4], <https://webrtcforthecurious.com/docs/05-real-time-networking/>.

frame requests and bandwidth updates directly, with no application involvement. The only existing opening for an application to touch this pipeline is **WebRTC Encoded Transform** [6], an existing Web API, introduced a few years ago, that lets a script observe and modify compressed frames in either direction as they pass between the browser’s encoder and the part of the pipeline that packages them for the network.

Although this internship did not implement or modify any audio or video codec, understanding a few of their key principles was necessary to justify why these two particular signals (a key-frame request and a bandwidth estimate, and not, say, a fixed schedule of key frames, or no bandwidth signal at all) are what a live encoder actually needs from the network. Section 5 looks briefly at how video and, more briefly, audio compression work.

5 Digital Audio and Video Encoding

Compression exists in the first place because raw, uncompressed video is enormous: a modest 1280×720 picture at 30 frames per second, in the chroma-subsampled format cameras typically capture, works out to over 300 megabits of data every second: dramatically more than most network connections (wired or wireless) can sustain for a single call, let alone leave room for anything else. Shrinking that by two or three orders of magnitude, down to a bitrate of a few megabits per second or less, is therefore not an optimization but a precondition

for real-time video existing on ordinary networks at all. Video compression is where this reduction is hardest won, so it is worth a closer look first.

An encoder shrinks a video stream mainly by not re-sending pixels that have not changed. The first frame of a stream, or of any point at which a decoder should be able to join, is coded entirely on its own, as a **key frame** (or *I-frame*): self-contained and decodable without any other information, but comparatively large. Every frame after it is usually coded as a **delta frame** (or *P-frame*) instead: rather than describing the picture from scratch, it describes how it differs from the frame before it, which is far cheaper to encode, at the cost of only making sense together with everything it was built on top of.

A stream is therefore a chain: a key frame followed by a run of delta frames each depending on the one before it, until the next key frame restarts the chain. If a single delta frame is lost, every delta frame after it is technically received but effectively meaningless, since each encodes a difference from a reference the decoder never got. The picture degrades and stays degraded until a new key frame arrives. This is why a key-frame request is a distinct signal from ordinary retransmission: retransmitting one lost packet cannot repair a broken chain, and the only way out is for the encoder to produce a new, self-contained frame on demand.

An encoder is also given a target **bitrate** (how many bits per second it is allowed to spend) and continuously trades that budget for quality: a higher target buys sharper, less blocky pictures at the same resolution and frame rate, a lower one forces coarser detail. Because that target reflects the current state of the network rather than the video content, it has to be supplied to the encoder from outside, and it can change from one moment to the next; an encoder that keeps producing more data than the network can currently carry causes the exact queueing and delay that a real-time call cannot tolerate. This is why a continuously updated bandwidth estimate, rather than a value negotiated once at the start of the call, is the second thing a real-time encoder cannot do without.

All of this is also computationally expensive, and video is usually where it shows: encoding it is demanding enough that most devices offload the work to a dedicated hardware encoder block to keep it affordable, but hardware support is limited, and a request that falls outside it forces the browser to fall back to a software encoder running on the general-purpose CPU instead, at a noticeably higher processing and power cost. Audio does not usually face this trade-off, encoding it is cheap enough on the CPU that it is normally done in software regardless, without needing hardware acceleration at all.

Audio has the same problem on a smaller scale: an uncompressed 48 kHz, 16-bit stereo signal, the format WebRTC captures by default, is still around 1.5 megabits per second, which is worth shrinking even though it is a fraction of what video requires. Opus, the codec WebRTC uses by default, routinely brings that down to well under a tenth of its raw size, compressing each successive

20-millisecond slice of audio and, like a video encoder, is given a bitrate target it must live within. It does not, however, build the same long, fragile dependency chains that video does, so losing one audio packet degrades a fraction of a second of sound rather than everything that follows it, and audio streams never need an explicit key-frame request.

An application that supplies its own already-compressed frames, as described in Section 8, is in the same position as an encoder: it is the one entity that can act on a key-frame request by producing a self-contained frame, and the one entity that can act on a bandwidth estimate by changing what it produces next. Any design that lets such an application inject frames into a peer connection has to deliver it these same two signals, or it will behave worse than the encoder it replaced.

6 Limitations of the Current Web Platform

The Web Platform already gives an application two of the pieces it would need to supply its own compressed media. **WebCodecs** [7] gives direct, low-level access to the browser’s own encoders and decoders. **WebAssembly** lets an application ship a custom codec, compiled from C, C++ or Rust, that the browser does not provide natively. Together, these already let a web application produce compressed audio or video entirely on its own.

What it cannot do is send that media over a real-time connection. **Encoded Transform** (Section 4) is the closest existing opening, but it only lets an application *modify* frames that the browser’s own encoder already produced. There is no way to *originate* a frame from outside and have it processed by a peer connection. And even where it is used, Transform sits at the wrong point in the pipeline to receive an encoder’s control signals: it is a filter placed between the encoder and the packetizer, whereas an application supplying its own compressed media needs to receive the key-frame requests and bandwidth updates that would ordinarily go to an encoder, since it is now effectively the encoder.

The only alternative available today is to carry compressed frames over a general-purpose transport (WebTransport or WebSocket) and reimplement packetization, congestion control and loss recovery in WebAssembly: an enormous duplication of logic that WebRTC has spent years getting right, and not a realistic option for production software.

The underlying capability already exists outside the browser: native applications and server-side relays (Selective Forwarding Units, or SFUs, the components most large conferencing systems are built around) have supplied their own encoder directly to the WebRTC library for years. What the Web Platform was missing was a way to expose that same capability safely to untrusted web code, and that is what this project introduces.

7 Use Cases

Two concrete cases motivated this work.

Custom codecs. An application built with WebCodecs and WebAssembly (Section 6) may have an encoder the browser does not: a newer standard codec, one tuned for screen content, a research codec, or a proprietary one. Today, such an encoder has nowhere to send its output; it needs the ability to inject its already-compressed frames directly into a peer connection.

Enterprise content delivery (eCDN). This case targets large conferences or broadcasts distributed to many clients sitting on the same local network, a company-wide town hall, for instance, watched by hundreds of employees on the same office network at once.

Rather than have every client pull the stream independently over the network’s outward-facing internet connection, an eCDN deployment uses *peer-assisted media forwarding*: a small number of clients inside the local network receive the stream once and relay it internally to the rest, over the much cheaper local link, so the shared entry point to the outside network is never overloaded, as shown in Figure 5.

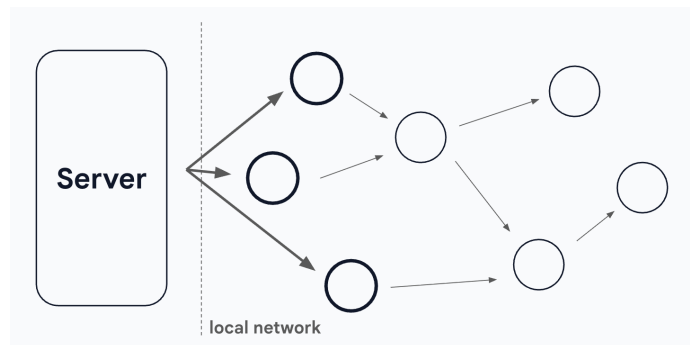


Figure 5: Peer-assisted media forwarding: the server outside the local network reaches only a handful of relaying clients (bold circles), which then forward the stream on to the rest over the cheaper local link, without the server ever addressing them directly.

Building this in the browser requires a way to feed encoded media straight into a peer connection: a relaying client extracts the compressed frames arriving on its inbound connection (using the existing Encoded Transform API, without decoding them) and injects those same frames, unchanged, into each outbound connection to the clients it serves.

Without this, a relay would have to decode the stream once and then re-encode it separately for every peer it forwards to: each of those extra encodes adds its own processing delay on top of the network path, working against the low latency real-time media depends on, and competes for the same limited

hardware encoder capacity discussed in Section 5, quickly forcing at least some of them into the much more expensive software fallback.

Forwarding the same compressed frames unchanged, by contrast, costs the relay no decoding, no re-encoding, and none of the latency that comes with either, no matter how many peers it serves.

8 Proposed Solution

The solution has two parts, designed and implemented together across both repositories that make up the browser’s real-time stack: the WebRTC library, and Chromium’s Blink layer, which is the component that implements the interfaces JavaScript actually sees.

Frame construction. An application first needs to be able to *build* a compressed frame object from a payload and some metadata, rather than only reading one out of an existing connection. I implemented constructors for this (`RTCEncodedAudioFrame` and `RTCEncodedVideoFrame`) so that an application using a WebCodecs encoder, for instance, can take the *chunk* it produces and turn it into a frame ready for injection. Both constructors take a single init object carrying the compressed payload alongside the metadata a standalone frame needs to make sense on its own, for video:

```
const videoFrame = new RTCEncodedVideoFrame({
  type: 'key',
  payloadType: 96,
  mimeType: 'video/VP8',
  rtpTimestampWithoutOffset: 54321,
  width: 640,
  height: 480,
  captureTime: performance.now(),
  contributingSources: [1234],
  data: buffer,
});
```

and, similarly, for audio:

```
const audioFrame = new RTCEncodedAudioFrame({
  contentType: 'speech',
  payloadType: 111,
  mimeType: 'audio/opus',
  rtpTimestampWithoutOffset: 54321,
  captureTime: performance.now(),
  contributingSources: [1234],
  audioLevel: 0.69,
  data: buffer,
});
```

The field named `rtpTimestampWithoutOffset` is a direct consequence of the point made in Section 4: RFC 3550 [9] requires the 32-bit timestamp actually carried on the wire to be offset by a value chosen at random for each sender, precisely so that an outside observer cannot correlate or replay streams by their timestamp. Every `RTCEncodedVideoFrame` or `RTCEncodedAudioFrame` that existed before this project was always read out of a live sender or receiver, where that offset is already baked into the timestamp it carries; this is the first time a frame can exist detached from any sender at all (an application can build one, hold onto it, or feed the same frame data into more than one connection), and at that point in its life there is no SSRC, and therefore no per-sender offset, for the timestamp to include yet. The frame instead stores the timestamp the application actually knows, without that offset, and it is the browser that adds the real sender's offset on top of it once the frame reaches a live `RTCRtpSender` for injection.

The Encoded Source API. The second part is the injection path itself: a new `createEncodedSource()` method on the sending half of a peer connection returns an object, living in a worker thread, that accepts compressed frames coming from the application. It does so through a *writable stream* and inserts them into the transport pipeline.

This object takes the position the browser's own encoder used to occupy, rather than sitting after it, which is what distinguishes it from Encoded Transform and lets the two features be used together on the same connection.

Two events on that object carry the control signals described in Sections 4 and 5 back to the application: one fires when the network requests a key frame, the other whenever the allocated bandwidth changes, so that the application's own encoder can adapt exactly as the browser's would have. In practice, the API is used across two execution contexts:

```
// main.js: Window context
const worker = new Worker('encoder-worker.js');
await sender.createEncodedSource(worker);

// encoder-worker.js: Worker context
self.onrtcsenderencodedsourcesource = (event) => {
  const encodedSource = event.encodedSource;
  const writer = encodedSource.writable.getWriter();

  encodedSource.onkeyframerequest = () => {
    writer.write(nextKeyFrame());
  };

  encodedSource.onbitrateinfochange = () => {
    console.log(encodedSource.allocatedBitrate);
    console.log(encodedSource.availableOutgoingBitrate);
  };
};
```

```
while (running) {  
  writer.write(nextFrame());  
}  
};
```

`createEncodedSource()` is called on the main thread, where the sender itself lives, but the object it hands over is only usable inside the worker: the browser dispatches a `rtcsenderencodedsources` event there once the source is ready, and it is the worker's `onkeyframerequest` and `onbitrateinfochange` handlers, together with a writer obtained from `encodedSource.writable`, that do the actual work of reacting to the network and supplying frames.

Running this loop in a worker rather than on the main thread is not a peculiarity of this API: it is standard practice for continuous media processing on the Web Platform, already followed by both WebCodecs and Encoded Transform, precisely because it keeps a task that runs for as long as the call lasts off the thread responsible for layout, rendering and user input.

The architecture that makes this work safely is a **proxy encoder**. Rather than bypassing the browser's encoding pipeline (which would silently break the bookkeeping, bandwidth accounting and activity checks that assume a real encoder is running), the design installs a stand-in encoder inside the WebRTC library itself, exposed to Blink through a new libwebrtc-level API rather than being built directly into Chromium.

From the pipeline's point of view, this proxy is a completely ordinary encoder: it receives the same key-frame requests and bandwidth allocations any encoder would, and it participates in statistics and bookkeeping like any other. In practice, though, it does no compression: it simply hands over the frame the application already supplied.

This gives the mechanism a clean, symmetric shape, shown in Figure 6: compressed frames flow *down*, from the application in Blink, across the browser's internal layer boundary, into the WebRTC library and onto the network, while the two control signals flow in the *opposite direction*, from the network, through WebRTC's proxy encoder, back across that same boundary, up to the application in Blink.

Because the proxy behaves exactly like a real encoder to everything around it, the rest of the pipeline needs no special-casing, and an application that does not use the feature sees no change in behaviour at all.

Keeping most of this mechanism inside WebRTC, rather than in Chromium-specific code, also matters beyond this one browser: because other browser engines already embed the same library as a dependency (Section 3), most of the work needed to support the Encoded Source API elsewhere is already done once it lands in WebRTC, leaving comparatively little engine-specific integration for each of them to add on top.

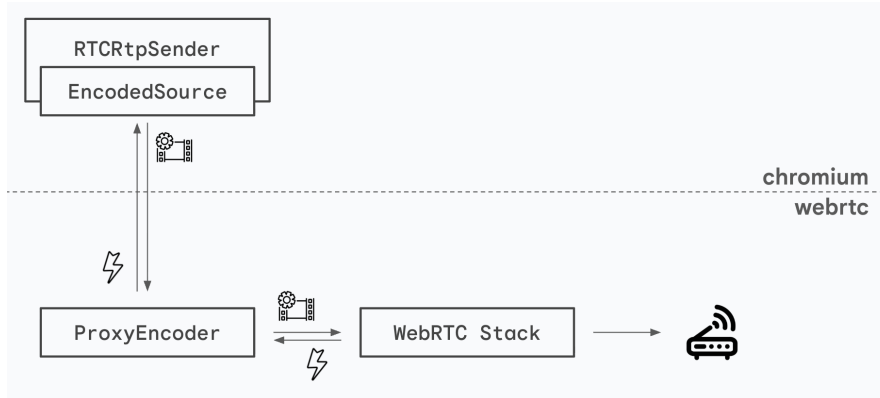


Figure 6: Architecture of the proposed solution. The Encoded Source lives inside the **RTCRtpSender**, on the Chromium side of the boundary between the two repositories; it exchanges compressed frames and control signals with the proxy encoder on the WebRTC side, which in turn exchanges them with the rest of the WebRTC stack on their way to and from the network.

The rest of WebRTC’s sending pipeline, however, still expects an ordinary, continuously running video or audio track feeding it raw frames at a steady cadence. That expectation is what drives frame timing, resolution and frame-rate adaptation decisions, and the activity checks mentioned above, and it does not go away just because the real content is coming from the application instead of a camera or microphone.

The implementation therefore keeps the pipeline supplied with a synthetic raw source in place of the missing capture device: a black video frame, or a buffer of silence, generated locally and fed into the pipeline right where captured media would otherwise have gone. When one of these placeholder frames reaches the proxy encoder, the encoder does not encode it: it discards it and returns, in its place, the already-compressed frame the application most recently supplied through the writable stream.

From the rest of the pipeline’s point of view, the encoder still received a raw frame and produced an encoded one, right on schedule.

The resulting design is documented in a public explainer [3], of which I am a co-author, and has received positive feedback in the W3C WebRTC Working Group.

9 Demonstration

Both the audio and video paths were validated with a working demonstration web application, run between a local pair peer connection objects.

On the sending side, a WebCodecs encoder captures and compresses the local

camera and microphone, the constructors from Section 8 turn each resulting chunk into a frame object, and the Encoded Source injects it into the `RTCPeerConnection`.

On the receiving side sits an unmodified peer: an ordinary `RTCPeerConnection` that decodes and renders the incoming stream exactly as it would any other call, with no knowledge that the frames arriving at it were ever anything other than normal WebRTC output.

The page itself displays both ends of this path at once: the local preview of what is being captured and encoded, and the remote video as actually received after the round trip, so that the two can be compared directly.

The two control signals introduced in Section 4 are not just logged: they actively drive the demonstration’s encoder. A key-frame request reaching the page forces the next call into the WebCodecs encoder to produce a key frame instead of a delta frame, and a bandwidth-allocation event reconfigures the encoder’s target bitrate to match, just as described in Section 5 for a browser-native encoder.

Both events, along with the values they carry, are also written out live on the page itself, so that the feedback loop is visible, not just effective.

Because the demonstration runs over a real, if local, network path, its bandwidth estimate reflects whatever that path can actually sustain, which on a local link is normally far more than a call would ever need. To exercise the bandwidth-adaptation side of the design deliberately, the available bandwidth between the two machines can be artificially constrained, which is enough to bring the sender’s bandwidth estimate down, trigger a `bitrateinfochange` event on the Encoded Source, and observe the demonstration’s encoder lower its target bitrate in response, live on the page.

10 Conclusion and Future Work

The Web Platform has spent two decades absorbing capabilities that were previously the preserve of native applications: real-time communication, low-level access to codecs, near-native compiled execution. What remained missing was the connection between them: an application could compress media itself, and it could communicate in real time, but it could not do both at once. This internship closed that gap: a web application can now construct a compressed audio or video frame from scratch and inject it into a live `RTCPeerConnection`, complete with the key-frame-request and bandwidth-allocation signals that let it behave like any other encoder in the pipeline, using a proxy-encoder architecture that keeps this new path indistinguishable, from the rest of the browser’s point of view, from an ordinary one.

The next step is standardization. The explainer that documents this design needs to be carried through the W3C WebRTC Working Group into a full specification, and the implementation adjusted to match that specification

wherever the two diverge, before other browser engines can adopt it. If that process succeeds, the contribution is not to one browser but to the Web Platform itself: closing a gap between what a web application can build and what native software and server-side infrastructure have been able to do for years.

A Glossary

Term	Meaning
Blink	The component of Chromium that implements the Web Platform interfaces JavaScript sees
Delta frame	Also called a P-frame; a compressed video frame coded as the difference from a previous frame, cheaper to encode than a key frame but undecodable without it
DTLS	Datagram TLS; the key-exchange protocol used by WebRTC over UDP
eCDN	Enterprise content delivery network; internal redistribution of a stream to reduce external bandwidth
Encoded Transform	Existing Web API allowing an application to observe and modify compressed frames in flight
GCC	Google Congestion Control; an algorithm that estimates available bandwidth from packet delay trends
ICE	Interactive Connectivity Establishment; the NAT-traversal procedure used by WebRTC
IETF	Internet Engineering Task Force; standardizes the wire protocols
Intent to Prototype / Intent to Ship	Chromium's process for shipping a new capability, from an experimental, flag-gated implementation to a public proposal reviewed by other browser vendors
Key frame	Also called an I-frame; a compressed video frame decodable on its own, as opposed to a delta frame coded as a difference from previous frames
Mojo	Chromium's interprocess-communication layer, used by a sandboxed renderer process to ask the privileged browser process to act on its behalf
NACK	Negative Acknowledgement; an RTCP message asking the sender to retransmit specific lost packets
Opus	The audio codec WebRTC uses by default
PLI / FIR	Picture Loss Indication / Full Intra Request; RTCP messages asking the sender for a key frame
Proxy encoder	A stand-in encoder installed inside the WebRTC library that receives key-frame requests and bandwidth allocations like a real encoder, but substitutes an application-supplied compressed frame instead of encoding one itself

Term	Meaning
Roll	The automated process that pulls the latest WebRTC commit into Chromium's own source tree
RTCP	The control and feedback channel accompanying RTP
RTP	Real-time Transport Protocol; carries media packets
Rule of Two	A Chromium security principle: code that parses untrustworthy input, is written in a memory-unsafe language, and runs with high privilege should satisfy at most two of the three
SDP	Session Description Protocol; the document format used to negotiate a connection's media and network parameters
SFU	Selective Forwarding Unit; a server that forwards compressed streams without decoding them
SRTP	Secure RTP; the encrypted form used by WebRTC
SSRC	Synchronization source; a randomly chosen identifier that lets several independent RTP streams share the same connection
STUN / TURN	Servers used to discover a public address, and to relay traffic when direct connection fails
V8	Chromium's JavaScript engine, which executes application script and, through generated bindings, calls into Blink
W3C	World Wide Web Consortium; standardizes the Web Platform interfaces
WASM	WebAssembly; portable binary format allowing compiled code to run in the browser
WebCodecs	Web API giving direct access to the browser's encoders and decoders

B List of Submitted Code Changes

All changes are public. WebRTC changes are reviewed at webrtc-review.googlesource.com, Chromium changes at chromium-review.googlesource.com.

List of merged changes for both projects:

- <https://webrtc-review.googlesource.com/q/status:merged+author:evil@chromium.org>
- <https://chromium-review.googlesource.com/q/status:merged+author:evil@chromium.org>

WebRTC

#	Link
1	https://webrtc-review.googlesource.com/c/src/+501580
2	https://webrtc-review.googlesource.com/c/src/+501780
3	https://webrtc-review.googlesource.com/c/src/+498801
4	https://webrtc-review.googlesource.com/c/src/+497100
5	https://webrtc-review.googlesource.com/c/src/+489340
6	https://webrtc-review.googlesource.com/c/src/+498600
7	https://webrtc-review.googlesource.com/c/src/+485761
8	https://webrtc-review.googlesource.com/c/src/+488340
9	https://webrtc-review.googlesource.com/c/src/+492780
10	https://webrtc-review.googlesource.com/c/src/+492400
11	https://webrtc-review.googlesource.com/c/src/+487340
12	https://webrtc-review.googlesource.com/c/src/+487360
13	https://webrtc-review.googlesource.com/c/src/+486860
14	https://webrtc-review.googlesource.com/c/src/+485781
15	https://webrtc-review.googlesource.com/c/src/+482900

Chromium

#	Link
1	https://chromium-review.googlesource.com/c/chromium/src/+8304692
2	https://chromium-review.googlesource.com/c/chromium/src/+8360971
3	https://chromium-review.googlesource.com/c/chromium/src/+8366130
4	https://chromium-review.googlesource.com/c/chromium/src/+8366404
5	https://chromium-review.googlesource.com/c/chromium/src/+8353464
6	https://chromium-review.googlesource.com/c/chromium/src/+8346748
7	https://chromium-review.googlesource.com/c/chromium/src/+8097283
8	https://chromium-review.googlesource.com/c/chromium/src/+8024752
9	https://chromium-review.googlesource.com/c/chromium/src/+8035461
10	https://chromium-review.googlesource.com/c/chromium/src/+8074218

C References

- [1] World Wide Web Consortium (W3C). <https://www.w3.org/>
- [2] Internet Engineering Task Force (IETF). <https://www.ietf.org/>
- [3] G. Urdaneta and L. Evi, *Explainer: WebRTC Encoded Source API and Encoded Frame constructors*. <https://github.com/guidou/webrtc-extensions/blob/main/encoded-source-explainer.md>
- [4] *WebRTC for the Curious*. <https://webrtcforthecurious.com/>

- [5] I. Grigorik, *High Performance Browser Networking*, chapter on WebRTC. <https://hpbnn.co/webrtc/>
- [6] W3C, *WebRTC Encoded Transform*. <https://w3c.github.io/webrtc-encoded-transform/>
- [7] W3C, *WebCodecs*. <https://w3c.github.io/webcodecs/>
- [8] The Chromium Projects, documentation. <https://www.chromium.org/chromium-projects/>
- [9] IETF, RFC 3550, *RTP: A Transport Protocol for Real-Time Applications*. <https://www.rfc-editor.org/info/rfc3550/>
- [10] IETF, RFC 9147, *Datagram Transport Layer Security Version 1.2*. <https://www.rfc-editor.org/info/rfc9147/>
- [11] IETF, RFC 3711, *The Secure Real-time Transport Protocol (SRTP)*. <https://www.rfc-editor.org/info/rfc3711/>
- [12] IETF, RFC 8825-8835, the WebRTC protocol suite.
- [13] IETF, RFC 8298 and related documents on congestion control for real-time media.
- [14] Chromium Developers, *Multi-process Architecture*. <https://www.chromium.org/developers/design-documents/multi-process-architecture/>
- [15] Chromium Security Team, *The Rule Of 2*. <https://chromium.googlesource.com/chromium/src/+/main/docs/security/rule-of-2.md>
- [16] Chromium Developers, *Intro to Mojo & Services*. https://chromium.googlesource.com/chromium/src/+/main/docs/mojo_and_services.md
- [17] The Chromium Projects, *Blink*. <https://www.chromium.org/blink/>
- [18] The V8 Project, *V8 JavaScript engine*. <https://v8.dev/>
- [19] Chromium Developers, *Blink-V8 bindings generator (bind_gen package)*. https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/bindings/scripts/bind_gen/README.md